A beam search for the air cargo palletization problem with weight distribution constraints

Ignacio Araya
Escuela de Ingeniería Informática, Pontificia Universidad Católica de Valparaíso,
Avenida Brasil 2950, Valparaíso, Chile
ignacio.araya@pucv.cl

October 28, 2026

Abstract

We address the palletization of air cargo into unit load devices (ULDs): a set of heterogeneous boxes must be loaded into ULDs of minimum total volume, where the ULDs have bevelled corners to fit the fuselage, a weight capacity and a floor edge for the cargo net locks, the boxes have allowed orientations and may be fragile, every box must be supported, and the center of gravity of each ULD must lie in a given area around its geometric center. We adapt a block-based beam search guided by a greedy algorithm (BSG), originally designed for the container loading problem, to this problem. The blocks are generated and placed so that the support and fragility constraints are met, blocks that cross a bevelled side are shifted to the nearest feasible position, and the center of gravity is taken into account in the value of the solutions, with a final translation of the load. The ULDs are loaded sequentially, and the total volume is then reduced by removing a ULD, inserting its boxes into the others and loading the rest into at most two smaller ULDs. On the 300 benchmark instances of Paquay et al. and with the same parameters in all the settings, the algorithm uses 2.4–3.8% less ULD volume than the state of the art at a comparable running time and 5.9–8.2% less with a few seconds per instance, with significant differences in all the settings, leaves less volume unloaded when the number of ULDs is limited, and almost none of its ULDs violates the center of gravity constraint, against 7–11% of the ULDs of the previous method.

Keywords: packing; air transport; three-dimensional bin packing; beam search; weight distribution

1 Introduction

Before a cargo aircraft takes off, the individual shipments are consolidated onto air cargo pallets and into containers, the so-called unit load devices (ULDs), which are then loaded into the aircraft. The shape of a ULD depends on its position in the aircraft: lower-deck containers have bevelled bottom corners and main-deck pallets and containers have bevelled top corners, so that they fit the fuselage. Building the ULDs is a daily operational task in air freight terminals, where it has to be solved for many flights in a short time, and every ULD saved is space and weight that can be sold or saved in fuel. At the same time, a ULD plan is only usable if it respects the practical constraints of air transport: the weight capacity of each ULD, the allowed orientations of the boxes, fragile boxes on which nothing can be placed, the stability of the load, and the weight distribution, since the center of gravity (CoG) of each ULD must lie in a given area around its geometric center (Paquay et al.,

2016). Building the ULDs is one of the subproblems of the air cargo load planning problem (Brandt and Nickel, 2019).

Assuming that the cargo consists of rectangular boxes, the problem is a three-dimensional Multiple Bin Size Bin Packing Problem (MBSBPP) in the typology of Wäscher et al. (2007), with non-cuboid bins and additional constraints. Paquay et al. (2016) proposed a mixed integer programming formulation of the problem, which can only solve small instances, and a set of instances derived from a real set of 562 boxes and the six ULD types of a Boeing 777. Paquay et al. (2018b) and Paquay et al. (2018a) proposed MIP-based and two-phase constructive heuristics for these instances. Recently, Heßler et al. (2025) extended the problem with further practical features (a floor edge for the cargo net locks, on which no box can be loaded, a substructure to raise the floor, and padding material to support the boxes) and proposed a randomized greedy search (RGS) based on a fast extreme-point insertion heuristic. The RGS uses 30% fewer ULDs than the heuristic of Paquay et al. (2018a) in a fraction of a second per instance, and it is, to the best of our knowledge, the state of the art for these instances.

In this paper we show that a different algorithmic paradigm, borrowed from the single container loading problem, yields considerably better ULD plans. The beam search guided by a greedy (BSG) algorithm (Araya and Riff, 2014; Araya et al., 2017) loads a container with *blocks* of identical or similar boxes placed in *maximal free spaces*, and selects the blocks with a beam search in which each node is evaluated by completing its partial solution with a greedy algorithm. It is among the best algorithms for the classical container loading problem (Araya et al., 2017, 2020), but the air cargo problem differs from it in several respects: the ULDs are not cuboids, the support of the boxes and the fragile boxes restrict how boxes can be stacked, the CoG of every ULD is constrained, several ULDs of different types must be loaded, and all the boxes must be loaded with as few ULDs as possible. Our contributions are the following:

- We adapt BSG (Araya and Riff, 2014; Araya et al., 2017), which builds the load with blocks of boxes (Fanslau and Bortfeldt, 2010) placed in maximal free spaces (Parreño et al., 2008) and evaluated with the VCS function (Araya et al., 2017), to load a ULD with bevelled corners, a floor edge and an optional substructure, the features introduced by Paquay et al. (2016) and Heßler et al. (2025). The blocks are generated so that every box inside a block is supported, and each block is placed in a free space only if the boxes of its base are supported according to the corner or area criterion of the problem and no box touches a fragile box. A block that crosses a bevelled side of the ULD is shifted along the length of the ULD to the nearest feasible position.

- We handle the CoG constraint in two ways: the value of a solution is penalized by the excess of the CoG deviation over the allowed one, so that the beam search takes it into account during the construction instead of in a post-processing step (Davies and Bischoff, 1999; Paquay et al., 2018a), and the final load of each ULD is translated as a whole to center its CoG, a simple form of the shift operators of Paquay et al. (2018a). To leave room for this translation, all the free spaces are anchored at the corner closest to the origin of the ULD, which keeps the load grouped.

- We embed the single-ULD algorithm into a sequential multi-ULD framework, which chooses the type of each ULD with the rule of Heßler et al. (2025), and reduce the total volume of the ULDs, the objective of the problem, with a single move that generalizes the elimination of bins in local search for bin packing (Faroe et al., 2003) and the repacking into smaller bins of Ortmann et al. (2010): a ULD is removed, its boxes are inserted into the other ULDs and the remaining ones are loaded into at most two smaller ULDs. When the number of ULDs

is limited, the boxes left unloaded are inserted into the loaded ULDs with the same insertion procedure, based on ruin and recreate (Schrimpf et al., 2000), and the construction is restarted loading first the boxes that were left out.

- We tune the parameters once with irace (López-Ibáñez et al., 2016) on a separate training set and evaluate the algorithm on the 300 instances of Paquay et al. (2018a) in the three settings considered by Heßler et al. (2025). With the same parameters in all the settings, our algorithm uses 2.9–7.1% less ULD volume than the RGS in the original setting, 3.8–8.2% less in the adapted one and 2.4–5.9% less with one ULD of each type, where it also leaves less volume unloaded. The smaller savings are obtained at a running time comparable to that of the RGS, and the differences are significant in all the settings. Almost none of its ULDs violates the CoG constraint, against 7–11% of the ULDs of the RGS. All the solutions are checked by an independent validator, and the code, the solutions and the scripts are publicly available.

The remainder of the paper is organized as follows. Section 2 defines the problem and Section 3 reviews the related literature. Section 4 describes the algorithm, Section 5 reports the computational experiments, and Section 6 concludes the paper.

2 Problem definition

We follow the problem definition of Paquay et al. (2016) with the extensions of Heßler et al. (2025). The length, width and height of a ULD correspond to the x, y and z axes, respectively; z is the vertical axis.

Boxes. A set I of rectangular boxes must be loaded. Each box $i \in I$ has a length l_i, a width w_i, a height h_i and a weight q_i. The boxes are placed orthogonally, and each box has a set of allowed orientations: for each of its three dimensions, the instance states whether the box can be placed with that dimension in the vertical direction (each allowed vertical dimension gives two orientations, rotated or not around the vertical axis). A box can be fragile, in which case nothing can be placed on top of it. The weight of a box is uniformly distributed, so that its CoG is its geometric center. The instance files of Paquay et al. (2018a) also flag very dense boxes; as Heßler et al. (2025), we do not use this flag.

ULDs. A set T of ULD types is available, in unlimited number or with a given number n_k of ULDs of each type $k \in T$. A ULD of type k is the bounding box $[0, L_k] \times [0, W_k] \times [0, H_k]$ from which up to four corners are cut by planes orthogonal to the x-z plane, to fit the fuselage of the aircraft. We represent each cut by a half-plane $\{(x, z) : a\,x + b\,z \leq c\}$ in the x-z plane that contains the usable space; a box is inside the ULD if it is inside its bounding box and its projection on the x-z plane satisfies all the cuts. The ULD type also defines its usable volume V_k, its weight capacity Q_k, the maximum deviations d_k^x and d_k^y of the CoG of the load from the geometric center in the length and width directions, and the maximum height d_k^z of the CoG. Following Heßler et al. (2025), the floor of the ULD may have an edge of width e for the cargo net locks: no box can be loaded over the edge below the height δ (the vertical edge offset). To load large boxes at the bottom, a substructure of height δ can be used, which raises the floor (without the edge) to the height δ; in that case no box can be loaded below the height δ.

Constraints. A loading plan assigns each loaded box to a ULD, an orientation and a position. It is feasible if:

1. each box is inside its ULD and, if the ULD has an edge, it does not overlap the edge below the height δ;
2. the boxes do not overlap;
3. the total weight of the boxes in each ULD of type k does not exceed Q_k;
4. each box whose base is not on the floor is supported: either the four corners of its base rest on other boxes (or on the floor or the substructure), or part of its base rests directly on other boxes and at least a proportion o of its base area rests directly on boxes or on padding material (Heßler et al., 2025); padding material can fill gaps of height at most $\hbar$ between the base of a box and the top of the boxes below;
5. no box, and no padding material, is placed on top of a fragile box;
6. the CoG $(\bar{x}, \bar{y}, \bar{z})$ of the load of each ULD satisfies $|\bar{x} - L_k/2| \leq d_k^x$, $|\bar{y} - W_k/2| \leq d_k^y$ and $\bar{z} \leq d_k^z$.

The original problem of Paquay et al. (2016) corresponds to $e = \delta = \hbar = 0$ and $o = 1$, that is, a box is supported only if the four corners of its base are supported. Paquay et al. (2018a) also allow the corners to rest on a bevelled wall of the ULD; we do not use this possibility, so our solutions are feasible for their definition too. Heßler et al. (2025) do not consider the vertical CoG constraint; we consider it in all the experiments. Moreover, both Paquay et al. (2018a) and Heßler et al. (2025) treat the horizontal CoG constraint as a soft constraint, since their solutions contain ULDs with the CoG out of range; we report the number of such ULDs for all the methods.

Objective. Following Paquay et al. (2016) and Heßler et al. (2025), the objective is to minimize the total volume of the used ULDs, that is, the unused volume. When the number of ULDs is limited, some boxes may be left unloaded, and the objective is first to maximize the loaded volume and then to minimize the total volume of the ULDs (Heßler et al., 2025). Since Paquay et al. (2018a) and Heßler et al. (2025) also report the number of ULDs, we report it as a secondary measure.

3 Related work

Three-dimensional packing problems with practical constraints have received much attention; we refer to Bortfeldt and Wäscher (2013) for a review of the constraints considered in container loading, to Zhao et al. (2016) for a comparison of container loading algorithms, and to Brandt and Nickel (2019) for a definition of the air cargo load planning problem and a review of its subproblems. We focus on the works closest to ours: air cargo palletization, packing with weight distribution constraints, and the heuristics on which our algorithm is based.

Air cargo palletization. Chan et al. (2006) developed a decision support system for loading air cargo pallets in a real case study. Paquay et al. (2016) formulated the problem considered in this paper as a mixed integer program, with non-cuboid ULDs, weight capacities, orientations, fragile boxes, support and CoG constraints, and solved small instances with a commercial solver. To solve larger instances, Paquay et al. (2018b) proposed three MIP-based constructive heuristics (relax-and-fix, insert-and-fix and fractional relax-and-fix), and Paquay et al. (2018a) a tailored two-phase

constructive heuristic: the first phase packs the boxes into identical bins with an extension of the extreme points, and the second one deals with the several ULD types; if necessary, the weight distribution of the best loading is then improved in a post-processing step with shift and jump operators that move or repack boxes. Heßler et al. (2025) extended the problem with the ULD edge, the substructure and the padding material, and proposed a randomized greedy search (RGS). Each ULD is loaded by repeatedly calling an insertion heuristic with several sorting criteria of the boxes and a random perturbation of the order; the insertion heuristic places each box at the first feasible extreme point (Crainic et al., 2008), uses an extended set of extreme points that can be moved to fill the space under the bevelled sides of the ULD, and accelerates the feasibility checks with a grid of cells. The best loading of a ULD is selected with a score that combines the volume utilization and the weight balance, and the ULDs are loaded sequentially, first those that are needed for the boxes that fit in the fewest ULD types. Their algorithm improves the results of Paquay et al. (2018a) by about 30% in a fraction of a second per instance. A related but different problem is the loading of the ULDs into the aircraft, which determines the position of each ULD to balance the aircraft, sometimes with several pickup and delivery points (Lurkin and Schyns, 2015; Mesquita and Sanches, 2026).

Weight distribution. Constraints on the CoG of the load arise in container loading (Davies and Bischoff, 1999), in pallet loading and in air cargo (Chan et al., 2006; Paquay et al., 2016), and they have also been studied in the three-dimensional knapsack problem (Baldi et al., 2012) and in the load-balanced bin packing problem (Trivella and Pisinger, 2016). Many heuristics handle them in a post-processing step, for instance by reordering and mirroring the walls of the load after the packing has been built (Davies and Bischoff, 1999) or by shifting and repacking boxes (Paquay et al., 2018a), or with a score that trades the weight balance against the volume utilization (Heßler et al., 2025). Paquay et al. (2018a) also consider the CoG during the construction, with a merit function that selects the extreme point that keeps the CoG closest to the center of the ULD. In our algorithm the CoG is taken into account in the value of the partial solutions that guides the beam search, and the whole load is translated at the end.

Heuristics for three-dimensional packing. Most heuristics for three-dimensional bin packing build the packing by placing one box at a time at a candidate position: the corner points of Martello et al. (2000), the extreme points of Crainic et al. (2008), or the maximal free spaces of Parreño et al. (2008). Parreño et al. (2008) and Parreño et al. (2010) embedded a maximal-space constructive heuristic in GRASP and variable neighbourhood descent, and Alvarez-Valdes et al. (2013) extended it with path relinking to the multiple bin size bin packing problem. Gajda et al. (2022) proposed a randomized constructive heuristic for a real container loading problem with weight distribution along the truck axles. For the single container loading problem, the best results have been obtained by tree search algorithms that place *blocks* of boxes instead of single boxes: Fanslau and Bortfeldt (2010) combined blocks of boxes with a partition-controlled tree search, Zhu and Lim (2012) proposed an iterative-doubling greedy lookahead algorithm, and Araya and Riff (2014) a beam search in which each node is evaluated by a greedy completion of its partial solution (BSG). Araya et al. (2017) introduced the VCS function to evaluate the blocks, which combines the volume of the block, its contact surface with the placed boxes and the container, the volume that the block makes unusable in the free space, and the number of boxes, Araya et al. (2020) extended BSG to a biobjective container loading problem that also maximizes the profit of the loaded boxes, and Guesser et al. (2026) improved its implementation with cache-friendly data structures, the memoization of the greedy completions and a two-stage evaluation function, obtaining the best results to date on the classical instances. Heßler et al. (2025) also tested the RGS on the container loading instances of

Bischoff and Ratcliff (1995) and reported that it lags behind BSG with VCS (Araya et al., 2017), which they attribute to the much larger number of boxes per container in those instances. To the best of our knowledge, block-based tree search has not been applied to air cargo palletization, where the boxes are highly heterogeneous, the ULDs are not cuboids, and the support, fragility and CoG constraints restrict the blocks that can be built and where they can be placed. The main purpose of this paper is to show how to handle these features and that, with them, block-based tree search also outperforms the state of the art on this problem.

Finally, the reduction of the volume and the insertion of the unloaded boxes in our multi-ULD framework are related to the local search heuristics that try to empty one bin at a time (Faroe et al., 2003), to the repacking of the contents of large bins into smaller ones in variable-sized bin packing (Ortmann et al., 2010), and to the ruin and recreate principle (Schrimpf et al., 2000).

4 Algorithm

The algorithm loads the ULDs one at a time. Each ULD is loaded by BSG, a beam search guided by a greedy algorithm, which builds the load with blocks of boxes placed in maximal free spaces (Section 4.1). The ULD-specific features (the bevelled corners, the edge and the substructure, and the support and fragility constraints) are handled in the generation and the placement of the blocks (Section 4.2), and the CoG constraint at several levels of the search (Section 4.3). Section 4.4 puts these components together to load one ULD, and Section 4.5 describes the multi-ULD framework.

4.1 Loading one ULD with BSG

Blocks. A block is a cuboid arrangement of boxes that is placed as a unit. Before loading a ULD, a set of blocks is generated from the boxes still available, as in Fanslau and Bortfeldt (2010) and Araya and Riff (2014): the blocks of one box are the boxes in their allowed orientations, and new blocks are obtained by joining two existing blocks along the x, y or z axis, so that every block has a guillotine structure. A joined block is kept only if its boxes are available, it fits in the bounding box of the ULD, the boxes fill at least a proportion $f_{\min}$ of its volume, and it is different from the blocks already generated (same dimensions and same boxes); the generation stops when $N_{\max}$ blocks have been generated. We use $f_{\min} = 0.98$, as in BSG (Araya and Riff, 2014), so that the blocks are almost solid, and $N_{\max} = 500$ (Section 5.1). Each block stores its weight and its CoG, computed recursively from the two blocks that form it, which allows us to evaluate the CoG of a load without visiting its boxes.

Free spaces and actions. The free space of the ULD is represented by the set of maximal free spaces, the largest cuboids that do not intersect the placed blocks (Parreño et al., 2008). When a block is placed, the free spaces that intersect it are replaced by their maximal parts outside the block, and the spaces too small for any remaining box are discarded. In Araya and Riff (2014), each space is anchored at its corner closest to a corner of the container, and the next space to fill is the one whose anchor corner is closest (in Manhattan distance) to the corresponding corner of the container, so that the load grows from all the corners towards the center. Here, instead, every space is anchored at its corner closest to the origin of the ULD and the next space is the one closest to the origin, so that the load grows from one corner and stays grouped: a grouped load leaves room to translate it, which is needed to center its CoG (Section 4.3), and when it does not fill the ULD it is not spread against several walls. Ties are broken by the larger volume. An action places a block that fits in the selected space at its anchor corner, which may be modified as explained in Section 4.2; a

block is a candidate only if its weight does not exceed the remaining weight capacity of the ULD. A state of the search is a partial loading of the ULD, with its free spaces and the remaining boxes.

Evaluation of the blocks. Each block b that fits in the selected space is evaluated with the VCS function of Araya et al. (2017):

$$\mathrm{VCS}(b) = V(b)\,\big(1-\mathrm{loss}(b)\big)^{\beta}\,\mathrm{CS}_p(b)^{\alpha}\,n(b)^{-\gamma}, \tag{1}$$

where $V(b)$ is the volume of the boxes of b, $\mathrm{loss}(b)$ is the proportion of the volume of the free space that cannot be filled with the remaining boxes after placing the block (estimated with one-dimensional knapsack problems on each dimension of the space), $\mathrm{CS}_p(b)$ is the proportion of the surface of the block in contact with the placed blocks and the walls of the ULD, where faces at a distance of at most a proportion p of the block dimension are considered in contact, and $n(b)$ is the number of boxes of the block.

Greedy and beam search. The greedy algorithm repeatedly selects the next free space and places the best evaluated block in it, until no block can be placed. The value of a complete loading is its volume utilization penalized by the CoG excess (Section 4.3). BSG (Araya and Riff, 2014) is a beam search in which the children of each node are generated with the best evaluated blocks for the selected space, and each child is evaluated by the value of its greedy completion. At each level, the w children with the best greedy values are kept, and a total of at most m children are generated, so that each node of the beam is expanded with about m/w blocks. The best complete solution found by any greedy completion is returned (Figure 1). BSG is run iteratively with increasing effort until the time limit t is reached (Araya et al., 2017): it starts with $w = 4$ and $m = 16$, and after each run w and m/w are multiplied by $\sqrt{2}$. The search also stops as soon as a solution loads all the available boxes with the CoG in range, since no better solution exists for that ULD. The implementation follows the optimizations proposed by Guesser et al. (2026) for BSG: the free spaces and the placed blocks are stored in contiguous arrays, the contact surface is computed only with the blocks near the selected space, the duplicate blocks are detected with a hash during the generation, and the greedy completions are memoized: the states visited by the complete greedy runs are stored (identified by a hash of their placed blocks), and when a greedy run reaches a known state, the rest of the solution is rebuilt from memory instead of evaluating the blocks again. In addition, the volume loss of a block is computed only when an upper bound of its evaluation can enter the best ones. None of these optimizations changes the result of the search.

4.2 ULD constraints

Bevelled corners. Each cut of a ULD is a half-plane $a_j x + c_j z \le d_j$ in the x-z plane, and the usable space is the intersection of the bounding box $[0, L] \times [0, W] \times [0, H]$ with all the cuts. A block $[x, x+\ell] \times [z, z+h]$ (in the x-z plane) satisfies cut j if and only if its corner that maximizes $a_j x + c_j z$ does, that is, if $a_j(x + \ell\,[a_j > 0]) + c_j(z + h\,[c_j > 0]) \le d_j$. A block placed at the anchor corner of a space may cross one of the cuts. In that case, the block is shifted along the x axis, inside the space, to the closest position that satisfies all the cuts: for a fixed z, each cut with $a_j > 0$ gives an upper bound and each cut with $a_j < 0$ a lower bound on x,

$$x \le \left\lfloor \frac{d_j - c_j(z + h\,[c_j > 0])}{a_j} \right\rfloor - \ell \quad (a_j > 0), \qquad x \ge \left\lceil \frac{d_j - c_j(z + h\,[c_j > 0])}{a_j} \right\rceil \quad (a_j < 0),$$

and the block is moved to the point of the resulting interval, intersected with the extent of the space, closest to the anchor; if the interval is empty, the block is not placed in the space. Since the cuts are

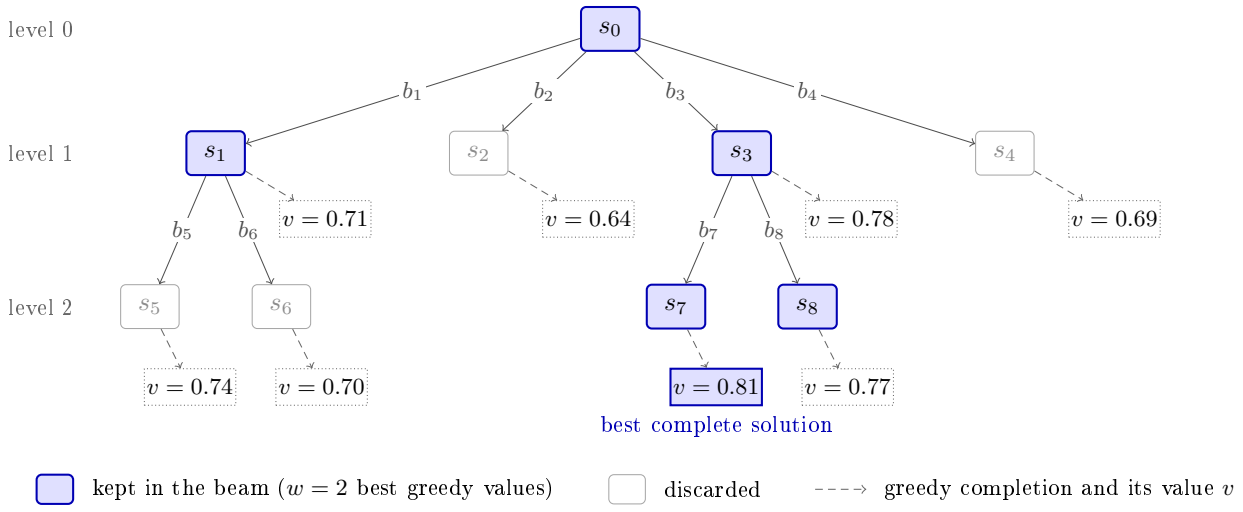


Figure 1: Two levels of BSG with $w = 2$ and $m = 4$. A state s_i is a partial loading of the ULD, obtained by placing block b_i in the selected free space of its parent. Every child is evaluated by the value v (Equation 3) of the complete loading built by the greedy algorithm from it (dotted boxes). The w children with the best values are kept in the beam and each of them is expanded with $m/w = 2$ blocks at the next level; the others are discarded. The best complete loading found by any greedy completion is returned.

orthogonal to the x-z plane, shifting the block along x keeps it on the same base, which is necessary for its support. The free spaces are still cuboids, so a space next to a cut may contain positions that are not usable; they are simply not used by any block.

Edge and substructure. With a floor edge of width e and offset δ, the initial free spaces are the space above the floor inside the edge, $[e, L-e] \times [e, W-e] \times [0, H]$, and the space above the height δ over the whole floor, $[0, L] \times [0, W] \times [\delta, H]$. With a substructure, the only initial space is the second one, and the floor inside the edge is raised to δ. A space above the edge has no support until boxes have been loaded inside it; instead of discarding it, it is kept aside (*dormant*) and reconsidered after the next block is placed. Each ULD with an edge is loaded twice, on its floor and on the substructure, and the best of the two loadings is kept; the blocks (single boxes and composite blocks) are generated for each of the two loadings, since the space in which they must fit is different.

Support and fragility. The support criterion of the problem (the four corners of the base, or the area criterion of Section 2) is checked box by box. Let R be the base of a box placed at height $z > 0$, and let the *direct* faces be the top faces of the placed boxes at height z and the *padding* faces those at a height in $[z - \hbar, z)$. The box is supported if each of the four corners of R lies on a direct face of a non-fragile box or, with the area criterion, if it overlaps some direct face and $A_d + A_p \geq o\,|R|$, where A_d and A_p are the areas of R covered by direct and padding faces of non-fragile boxes and o is the minimum supported proportion ($o = 0.9$ in the adapted settings); the raised floor of the substructure is also a direct face. The box is infeasible if R overlaps a direct face of a fragile box, and the padding faces of fragile boxes give no support. Inside a block, the support is guaranteed by construction: two blocks are joined along z only if the lower block has a complete top face without fragile boxes and the upper block fits on it, so that every box of the upper block has its whole base,

and in particular its four corners, on non-fragile boxes. Therefore, when a block is placed on the base of the selected space, only the boxes at its bottom are checked. Since the corner criterion allows gaps under a box, we also forbid placing a fragile box in such a gap if it would touch the base of the box above. If no block can be placed in a space because of the support, the space is discarded or, with an edge, kept dormant.

4.3 Center of gravity

The CoG constraint is taken into account in two ways, besides the anchoring of the spaces at the origin.

Translation of the load. Once a ULD has been loaded, the whole load can be translated horizontally without changing its feasibility, as long as every block stays inside the ULD and its cuts and, if the ULD has an edge, the blocks below the height δ stay inside the edge and the blocks on the substructure stay on it. These conditions define an interval of feasible translations in x and in y: a block $[x_1, x_2] \times [y_1, y_2] \times [z_1, z_2]$ allows $\Delta x \in [-x_1, L - x_2]$ and $\Delta y \in [-y_1, W - y_2]$, intersected, for Δx, with the bounds that the cuts give (as above, with z fixed) and, if $z_1 < \delta$ or the block lies on the substructure, with $[e - x_1, L - e - x_2]$ and $[e - y_1, W - e - y_2]$. The intervals of the load are the intersections over its blocks, so they are updated in constant time when a block is placed. The final load of each ULD is translated by $\Delta x = \min\big(\max(L/2 - \bar{x}, \Delta x^{\min}), \Delta x^{\max}\big)$, and Δy in the same way, the feasible amount that brings its CoG $(\bar{x}, \bar{y})$ closest to the center of the ULD.

CoG excess. Let $(\bar{x}, \bar{y}, \bar{z})$ be the CoG of a loading after the best feasible translation $(\Delta x, \Delta y)$. Its CoG excess is

$$\varepsilon = \frac{\big(|\bar{x} + \Delta x - L/2| - d^x\big)^+}{d^x} + \frac{\big(|\bar{y} + \Delta y - W/2| - d^y\big)^+}{d^y} + \frac{(\bar{z} - d^z)^+}{d^z}, \tag{2}$$

where $(a)^+ = \max(a, 0)$. The CoG constraint is satisfied if and only if $\varepsilon = 0$.

Value of a solution. The value of a (partial or complete) loading with volume utilization u is

$$v = \frac{u}{1 + \lambda\,\varepsilon}, \tag{3}$$

where $\lambda \geq 0$ is a parameter. This value is used to compare the greedy completions in BSG and to select the best loading of a ULD. A multiplicative penalty keeps the value of any non-empty loading positive, so that an unbalanced loading is still preferred to an empty ULD. With a large λ, the CoG constraint becomes almost a hard constraint; with $\lambda = 0$, it is ignored.

The weight and the moments of the partial loading and its interval of feasible translations are stored in each state, so that the CoG excess of a greedy completion is computed in constant time per placed block. Penalizing the CoG excess also in the evaluation of each candidate block had no consistent effect in our experiments, and it is not used.

4.4 Loading a ULD

The components of Sections 4.1–4.3 are combined in LoadULD (Algorithm 1), which loads a ULD of type k with the available boxes B. If the ULD has an edge, two loadings are built, one on the floor and one on the substructure (line 2). For each of them, the blocks that fit in that space are generated (line 3), BSG builds a loading within the time limit τ (line 4), the loading is translated

to center its CoG (line 5), and the one with the larger value v (Equation 3) is kept (line 6). The procedure is also used to complete a partial loading, keeping its boxes at their positions, which the multi-ULD phases below need.

Algorithm 1 LoadULD(k, B): load a ULD of type k with the available boxes B, with a time limit τ for each run of BSG (t, or $0.1\,t$ in the phases of Section 4.5).

1: $S^* \leftarrow$ empty loading
2: **for** $s \in \{\text{floor}, \text{substructure}\}$ **do** ▷ the substructure only if the ULD has an edge
3: generate the blocks of B that fit in a ULD of type k loaded on s
4: $S \leftarrow \text{BSG}(k, s, B, \tau)$
5: translate S to center its CoG
6: **if** $v(S) > v(S^*)$ **then** $S^* \leftarrow S$
7: **return** S^*

4.5 Loading several ULDs

Algorithm 2 outlines the multi-ULD framework. A sequential construction, Construct (Algorithm 3), loads one ULD at a time (line 3); when the number of ULDs is limited, the boxes left out are inserted into the loaded ULDs and the construction may be restarted (lines 5–7); finally, a reduction phase lowers the total volume of the ULDs, the objective of the problem (Reduce, Algorithm 5, line 9). The phases after the first loading of each ULD load many ULDs again, so BSG is given a shorter time limit in them: $0.1\,t$ in our experiments, both in Insert and in the reduction phase and when the candidate types of a ULD are compared during the construction.

Sequential construction. In Construct (Algorithm 3), the type of the next ULD is selected as in Heßler et al. (2025) (line 3): among the ULD types into which the boxes that fit in the fewest available types fit, the type with the largest total volume of the remaining boxes that fit in it is selected, ties being broken by the larger ULD volume. A box fits in a ULD type if its weight does not exceed the capacity and, in some allowed orientation, it can be placed alone on the floor inside the edge or on the substructure. When this rule leaves several candidate types (those that contain the boxes with the fewest options), the type is chosen with short loadings and only the chosen one is loaded with the full time limit: each candidate is loaded with LoadULD with the time limit $0.1\,t$, the one that maximizes $L - \theta V$ is chosen (line 5), where L is the loaded volume and V the volume of the ULD, unless it leaves the CoG out of range and another candidate does not, and the chosen type is loaded again from scratch with the time limit t (line 7); the short loadings are discarded. With $\theta = 0$ this is the type that loads the most volume; a small $\theta > 0$ prefers a smaller ULD when it loads almost the same volume. When the number of ULDs is limited, some boxes may only fit in the selected type among the ULDs still available. If the loading leaves some of them out (line 8), the ULD is loaded again with these boxes first, and the load is then completed with the other boxes, keeping the first ones at their positions.

Insertion of boxes into the loaded ULDs. The phases after the construction rely on the same procedure, Insert (Algorithm 4), which inserts a pool of boxes into the ULDs already loaded, without adding ULDs, by ruin and recreate (Schrimpf et al., 2000). For $\rho = 0.3$, 0.6 and 0.9, in this order, a proportion ρ of the loaded volume of each ULD is removed, taking the boxes with nothing on top of them from the highest one down, so that the remaining load is still feasible; then the ULDs, in decreasing order of free volume, are completed with LoadULD with the removed boxes

Algorithm 2 Multi-ULD framework.

1: $P \leftarrow \emptyset$ ▷ boxes to load first (restarts)
2: **repeat**
3: $(\mathcal{S}, B) \leftarrow$ CONSTRUCT(P) ▷ loaded ULDs, unloaded boxes
4: **if** $B \neq \emptyset$ **then**
5: INSERT$(\mathcal{S}, B)$ ▷ unloaded boxes (limited ULDs)
6: keep $(\mathcal{S}, B)$ if it is the first or leaves less volume unloaded; $P \leftarrow P \cup B$
7: **until** $B = \emptyset$ or three restarts
8: $(\mathcal{S}, B) \leftarrow$ the kept solution
9: REDUCE$(\mathcal{S})$ ▷ reduction of the volume
10: **return** $\mathcal{S}$, B

Algorithm 3 CONSTRUCT(P): sequential construction; the boxes of P are loaded first (restarts).

1: $B \leftarrow I$; $\mathcal{S} \leftarrow \emptyset$ ▷ remaining boxes, loaded ULDs
2: **while** $B \neq \emptyset$ **and** some available ULD type fits a box of B **do**
3: $K \leftarrow$ candidate ULD types for B (Heßler et al., 2025)
4: **if** $|K| > 1$ **then**
5: $k \leftarrow \arg\max_{k' \in K}(L - \theta V)$ of LOADULD(k', B), time limit $0.1\,t$
6: **else** $k \leftarrow$ the type in K
7: $S \leftarrow$ LOADULD(k, B) with time limit t
8: **if** the ULDs are limited and S leaves out boxes of P or that only fit in its type **then**
9: load these boxes first and complete S with the other boxes of B
10: $\mathcal{S} \leftarrow \mathcal{S} \cup \{S\}$; remove the boxes of S from B
11: **return** $(\mathcal{S}, B)$

and the pool, keeping the remaining boxes at their positions. A ULD whose new loading has the CoG out of range when the old one had not keeps its old loading. The new solution is accepted if the volume of the pool decreases, and the procedure stops as soon as the pool is empty. Reloading each ULD from scratch with its boxes and the pool, or giving priority to the boxes of the pool, did not improve the results at the same running time.

Unloaded boxes and restarts. When the number of ULDs is limited and some boxes remain unloaded after the construction, they are inserted into the loaded ULDs with INSERT (line 5). If some are still left out, the construction is restarted, and the boxes left out in the previous constructions are also loaded first, into the first ULD in which they fit (the set P in line 8 of Algorithm 3). This handles boxes that fit in several types but conflict with each other, for instance two large boxes that fit only in the largest types and cannot share a ULD. The solution with the smallest unloaded volume is kept (line 6), with at most three restarts.

Reduction of the volume. The construction tends to use the largest ULDs, and its last ULDs are often far from full. The reduction phase replaces ULDs by cheaper ones with a single move that generalizes the elimination of ULDs. The ULDs are considered in decreasing order of lost volume (the volume of the ULD minus the volume of its boxes). The boxes of the selected ULD S are freed and inserted into the other ULDs with INSERT (Algorithm 5, line 3). If all of them are inserted, S is eliminated (line 4). Otherwise, NEWULDS (Algorithm 6, line 5) loads the remaining boxes into one or two new ULDs whose total volume is smaller than that of S: the combinations of one or two

Algorithm 4 INSERT$(\mathcal{S}, B)$: insert the boxes of the pool B into the loaded ULDs $\mathcal{S}$ by ruin and recreate.

for $\rho \in \{0.3, 0.6, 0.9\}$ **do**
  if $B = \emptyset$ **then return**
  $\mathcal{S}' \leftarrow \mathcal{S}$; $B' \leftarrow B$
  for $S \in \mathcal{S}'$ in decreasing order of free volume **do**
    $R \leftarrow$ boxes removed from S, from the highest uncovered one down, until a proportion ρ of its loaded volume is removed
    $S^\circ \leftarrow$ complete $S \setminus R$ with LOADULD on the boxes $R \cup B'$ (time limit $0.1\,t$)
    if S° is feasible **and** (the CoG of S° is in range **or** that of S is not) **then**
      $B' \leftarrow (B' \cup R) \setminus S^\circ$; replace S by S° in $\mathcal{S}'$
    if $B' = \emptyset$ **then break**
  if $V(B') < V(B)$ **then** $\mathcal{S} \leftarrow \mathcal{S}'$; $B \leftarrow B'$

available types with enough volume and weight capacity, in which every remaining box fits, are tried in increasing order of total volume, loading the larger ULD first with all the remaining boxes and the second one with the rest. The first combination that loads all the boxes, without leaving a CoG out of range when the CoG of S was not, replaces S; otherwise the changes made by INSERT are undone and the next ULD is tried (line 7). After every change the phase starts again, and it stops when no ULD can be changed. Every change reduces the total volume of the ULDs, so the phase terminates; a change may increase the number of ULDs, when one ULD is replaced by two smaller ones.

Algorithm 5 REDUCE$(\mathcal{S})$: reduction of the total volume of the loaded ULDs $\mathcal{S}$.

repeat
  for $S \in \mathcal{S}$ in decreasing order of lost volume **do**
    $B' \leftarrow$ boxes of S; INSERT$(\mathcal{S} \setminus \{S\}, B')$
    if $B' = \emptyset$ **then** remove S from $\mathcal{S}$; **break**
    $\mathcal{N} \leftarrow$ NEWULDS(S, B')
    if $\mathcal{N} \neq$ failure **then** replace S by $\mathcal{N}$ in $\mathcal{S}$; **break**
    undo the changes of INSERT
until no ULD was changed

5 Computational experiments

This section evaluates the proposed algorithm on the benchmark of Paquay et al. (2018a) and on the two adapted variants introduced by Heßler et al. (2025), and compares it instance by instance with the randomized greedy search (RGS) of Heßler et al. (2025), which, to the best of our knowledge, reports the best results on these instances. Section 5.1 describes the instances, the implementation, the parameter tuning and the evaluation protocol. Section 5.2 reports the main results, Section 5.3 compares them with a lower bound, Section 5.4 analyzes the running time, Section 5.5 studies the contribution of each component and Section 5.6 the trade-off between the number of ULDs and the weight distribution.

Algorithm 6 NewULDs(S, B'): load the boxes B' left over from the ULD S into one or two new ULDs of smaller total volume.

for each set C of one or two available ULD types, in increasing order of total volume $V(C)$ **do**
  if $V(C) \geq V(S)$ **then return** failure
  if $V(C) < V(B')$ **or** the capacity of C is below the weight of B' **or** some box of B' fits in no type of C **then continue**
  $R \leftarrow B'$; $\mathcal{N} \leftarrow \emptyset$
  for $k \in C$, the larger type first, while $R \neq \emptyset$ **do**
    $S_k \leftarrow$ LoadULD(k, R) (time limit $0.1\,t$)
    if S_k is empty or infeasible, or its CoG is out of range and that of S is not **then break**
    $R \leftarrow R \setminus S_k$; $\mathcal{N} \leftarrow \mathcal{N} \cup \{S_k\}$
  if $R = \emptyset$ **then return** $\mathcal{N}$
return failure

5.1 Experimental setup

Instances. We use the 300 final instances of Paquay et al. (2018a): ten groups of 30 instances with $|I| \in \{10, 20, \ldots, 100\}$ boxes sampled from a real set of 562 boxes, highly heterogeneous, with weights, allowed vertical orientations and fragile (non-stackable) boxes. Six ULD types of a Boeing 777 are available in unlimited number: three lower-deck containers (types 1–3, with up to two bottom corners cut to fit the fuselage) and three main-deck pallets and containers (types 4–6, with up to two top corners cut). Each ULD type has a weight capacity and a maximum deviation of the center of gravity (CoG) from its geometric center in the length and width directions, and a maximum height of the CoG (Paquay et al., 2016). Following Heßler et al. (2025), we consider three settings:

- **Paquay**: the original instances. A box is supported if its four bottom corners rest on non-fragile boxes or on the floor, and there is no padding material. For these instances, Heßler et al. (2025) set the maximum CoG deviation to 5% of the half length and half width of the ULD, which is stricter than the deviations of the instance file (10–20% of the half dimensions); we use the same limit.
- **Adapted**: the ULD floor has an edge of width $e = 10\,\text{mm}$ for the cargo net locks, on which no box can be loaded below the height $\delta = 10\,\text{mm}$; a substructure of height δ can be used to raise the floor. A box is supported if its four bottom corners are supported or if part of its base is directly supported and at least 90% of its base is supported directly or by padding material of height at most 10 mm. The allowed CoG deviation is 10% of the half length and half width of the ULD. These are the default problem parameters of Heßler et al. (2025, Table 2).
- **One**: the adapted instances with only one ULD of each type, so that some boxes may remain unloaded. As in Heßler et al. (2025), solutions are compared lexicographically: first the loaded volume, then the number of ULDs.

In the three settings the objective is the total volume of the ULDs (Paquay et al., 2016; Heßler et al., 2025) (in the setting *One*, after the loaded volume); we also report the number of ULDs. Our algorithm uses the same parameters and components in the three settings; only the problem data (edge, support criterion, CoG limits and number of ULDs) change.

Implementation. The algorithm is implemented in C++ (compiled with g++ 13.3, `-O3`). Each run uses a single thread; the experiments were carried out on a server with two Intel Xeon E5-2620

v4 processors (8 cores each, 2.1 GHz, 3.0 GHz in turbo mode, hyper-threading disabled) and 62 GB of RAM, running 14 instances in parallel. According to the single-thread rating of PassMark (PassMark Software), one core of the Apple M2 Pro used by Heßler et al. (2025) is about 2.5 times faster than one core of our processor (4105 against 1624). The time limit t is the time of each run of BSG in the construction, excluding the generation of the blocks; in the improvement phases (Insert and the reduction), which load many ULDs again, each run of BSG is limited to $0.1\,t$ (Section 5.4 compares it with t). A search also stops as soon as it loads all the available boxes with the CoG in range. We report results for $t = 0.1$ s, whose total time per instance is about that of the RGS once the difference in processor speed is taken into account, and for $t = 1$ s. Since a ULD is loaded several times (the candidate types, with and without the substructure, and in the improvement phases), the total time per instance is several times t, and we report it separately. The code, the instances, the solutions and the scripts that produce all the tables are available at https://github.com/rilianx/bsg-uld.

Parameter tuning. The parameters of the algorithm were tuned once with irace (López-Ibáñez et al., 2016), as Paquay et al. (2018a) did for their heuristic, on a separate training set, and are used unchanged in the three settings and for both time limits. The training set consists of 150 preliminary instances of Paquay et al. (2018a) (30 instances with 30, 40, 50, 60 and 100 boxes), disjoint from the final ones, each one in the three settings (450 runs, presented to irace in blocks of the three settings of an instance), with $t = 0.1$ s, $N_{\max} = 500$ blocks and a budget of 6000 runs; no configuration is discarded before 21 instances. $N_{\max}$ was fixed before the tuning, because when irace could choose it, it chose the largest value and the running time grew. With the final parameters and $t = 0.1$ s, no other value of $N_{\max}$ is better in the three settings: 1000 blocks reduce the volume of the ULDs by one point in the Paquay setting but increase it by one point in the adapted one, with 7–12% more time, and 2000 blocks take 40–70% more time without reducing it (one run with 250, 1000 and 2000 blocks). The cost of a run is

$$\frac{V_{\text{ULD}} + 100\,V_{\text{unl}} + 150\,\text{m}^3 \cdot n_{\text{CoG}}}{V_{\text{boxes}}},$$

where V_{ULD} is the volume of the ULDs, V_{unl} the volume left unloaded, n_{CoG} the number of ULDs whose CoG is out of range and V_{boxes} the volume of the boxes of the instance. The two penalties are much larger than any change in the volume of the ULDs that the algorithm can make by changing one ULD (the largest ULD has $31.8\,\text{m}^3$): an unloaded box of $1\,\text{m}^3$ costs as much as $100\,\text{m}^3$ of ULDs and a ULD with the CoG out of range as much as $150\,\text{m}^3$, so that irace does not trade boxes left out or CoG violations for volume, and the volume of the ULDs decides among configurations that load all the boxes with the CoG in range. The values of a preliminary tuning were the initial configuration. Table 1 lists the parameters; the four elite configurations returned by irace agree on a small λ (1.0–1.7), a large γ (0.77–0.97) and θ between 0.18 and 0.32. The tuning was carried out with an earlier version of the improvement phases (with the time limit t and two more stages in Insert); the later changes, described in Section 4.5, mainly reduce the running time, and the parameters were not tuned again.

Validation and comparison. Every solution is written to a file (position and orientation of each box) and checked by an independent validator, which does not share code with the solver: each box is loaded at most once, in an allowed orientation, inside its ULD and its cuts, outside the floor edge below δ, without overlapping other boxes, supported according to the setting and not on top of a fragile box, and the weight capacity of each ULD is respected. The validator also counts the ULDs whose CoG is out of the allowed deviation in the length or width direction (CoG_{xy}) and above

Table 1: Parameters tuned with irace (training instances of the three settings, $t = 0.1\,\mathrm{s}$).

Parameter	Description	Range	Value
λ (`cog_penalty`)	penalty of the CoG excess in the value of a solution	$[1, 50]$	1.404
$N_{\max}$ (`max_bl`)	maximum number of blocks	fixed	500
α	exponent of the contact surface (VCS)	$[0, 5]$	0.514
β	exponent of the volume loss (VCS)	$[0, 5]$	4.750
γ	exponent of the number of boxes (VCS)	$[0, 1]$	0.773
p	tolerance of the pseudo contact surface (VCS)	$[0, 0.1]$	0.088
θ (`theta`)	weight of the ULD volume in the choice of the type	$[0, 0.6]$	0.232

the maximum height (CoG_z, a constraint of Paquay et al. (2016) that Heßler et al. (2025) do not consider). All the 9000 solutions reported in this section are feasible. The comparison with Heßler et al. (2025) uses their instance-by-instance results (Tables 8, 10 and 12 of their online appendix). For each instance we count whether our solution is better (smaller total volume of the ULDs or, in the setting *One*, more loaded volume or, with the same loaded volume, smaller total volume of the ULDs), equal or worse, and we test the difference in the total volume of the ULDs with the Wilcoxon signed-rank test. The total volume of the ULDs of the RGS is computed from the type of each ULD (Tables 9, 11 and 13 of their appendix) and, in the setting *One*, its unloaded volume from the utilization of each ULD, which is rounded to 0.1%; we therefore consider two loaded volumes equal if they differ by less than $0.05\,\mathrm{m}^3$. Each configuration is run five times; we report the mean and the standard deviation.

5.2 Results

Table 2 summarizes the results. In the three settings, our algorithm uses less ULD volume than the RGS of Heßler et al. (2025): with $t = 0.1\,\mathrm{s}$, at a running time comparable to that of the RGS, 2.9% less in the Paquay setting, 3.8% less in the adapted one and 2.4% less with one ULD of each type, where it also leaves less volume unloaded; with $t = 1\,\mathrm{s}$, 7.1%, 8.2% and 5.9% less. Instance by instance, its solutions are better in 182.8 instances and worse in 51.8 in the Paquay setting with $t = 1\,\mathrm{s}$ (184 and 44.6 in the adapted one, 171 and 36 with one ULD of each type), and the differences in volume are significant in all the cases (Wilcoxon signed-rank test, $p \leq 1.2 \cdot 10^{-4}$ with $t = 0.1\,\mathrm{s}$ and $p < 10^{-12}$ with $t = 1\,\mathrm{s}$). The total cost of the ULDs given in the instance files decreases in the same way (8.1%, 10.0% and 6.3% with $t = 1\,\mathrm{s}$). At most 2.2% of our ULDs violate the CoG constraint in the length or width direction, and none in the Paquay and adapted settings with $t = 1\,\mathrm{s}$, whereas 7–11% of the ULDs of the RGS have the CoG out of range.

Since the objective is the volume, our algorithm replaces large ULDs by smaller ones whenever this saves volume, even if it uses more ULDs: with $t = 1\,\mathrm{s}$ it uses 2–6% more ULDs than the RGS (732.6, 689.0 and 745.0 against 708, 653 and 729), and with $t = 0.1\,\mathrm{s}$ 7–11% more. In the setting with one ULD of each type, it leaves 12 boxes ($17.8\,\mathrm{m}^3$) unloaded with $t = 1\,\mathrm{s}$ and 15.4 boxes ($20.0\,\mathrm{m}^3$) with $t = 0.1\,\mathrm{s}$, against 24 boxes ($25.9\,\mathrm{m}^3$) for the RGS.

The results are stable: the total volume of the five runs of each configuration varies by less than 0.2%. The running time is discussed in Section 5.4.

Figure 2 shows a solution in the setting with one ULD of each type: our algorithm uses three ULDs, as the RGS, but replaces the main-deck ULD of type 4 ($20.0\,\mathrm{m}^3$) used by the RGS by a lower-deck container of type 1 ($5.3\,\mathrm{m}^3$), and loads the type 6 almost completely with the help of the substructure.

Table 2: Results on the 300 final instances (mean ± standard deviation over five runs). TPH: two-phase heuristic of Paquay et al. (2018a); RGS: randomized greedy search of Heßler et al. (2025). Vol.: total volume of the ULDs (m^3); B/W: number of instances in which our solution is better/worse than the RGS in its objective (loaded volume, then volume of the ULDs). Unloaded: boxes (m^3). CoG_{xy}, CoG_z: number (percentage) of ULDs with the CoG out of range. Time: mean time per instance (s).

Setting	Method	Vol. (m^3)	vs RGS	B/W	ULDs	Unloaded	CoG_{xy}	CoG_z	Time
Paquay	TPH	–	–	–	1010	0	130 (12.9%)	–	≤ 12
	RGS	18 648			708	0	79 (11.2%)	–	≤ 0.5
	BSG, $t = 0.1$ s	18 106 ± 17	−2.9%	158 / 74.4	777.6	0	7.8 (1.0%)	4.2	0.56
	BSG, $t = 1$ s	17 317 ± 4	−7.1%	182.8 / 51.8	732.6	0	0 (0%)	1.4	3.5
Adapted	RGS	16 843			653	0	47 (7.2%)	–	≤ 1.3
	BSG, $t = 0.1$ s	16 203 ± 25	−3.8%	151.4 / 62.8	725.8	0	3 (0.4%)	3	0.77
	BSG, $t = 1$ s	15 461 ± 6	−8.2%	184 / 44.6	689.0	0	0 (0%)	0	5.2
One	RGS	16 166			729	24 (25.9)	74 (10.1%)	–	≤ 1.1
	BSG, $t = 0.1$ s	15 773 ± 16	−2.4%	132.6 / 51.2	776.6	15.4 (20.0)	17 (2.2%)	11	0.89
	BSG, $t = 1$ s	15 207 ± 6	−5.9%	171 / 36	745.0	12 (17.8)	10.6 (1.4%)	8.8	6.1

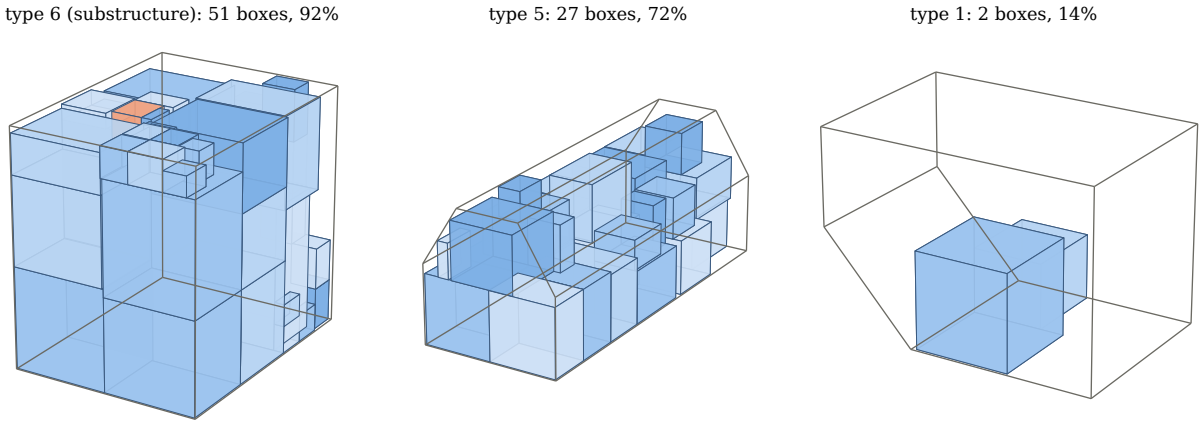


Figure 2: Solution of the instance with 80 boxes number 8 in the setting with one ULD of each type ($t = 0.1$ s): three ULDs (number of boxes and volume utilization) with a total volume of 60.3 m^3, against 75.0 m^3 for the ULDs of types 5, 6 and 4 of the RGS. Types 5 and 1 have bevelled corners; the type 6 uses the substructure. The load of each ULD has been translated to center its CoG, which is within range in the three ULDs.

Results by instance size. Tables 3 and 4 report the total volume and the number of ULDs by instance size. With $t = 1\,\mathrm{s}$ our algorithm uses less volume than the RGS in every group of every setting, between 0.2% and 16.6% less. In the instances with 10 boxes, which fit in one ULD, it uses more ULDs than the RGS (42–43 against 30) but less volume, since it splits the boxes into two smaller ULDs when their total volume is smaller than that of one large ULD.

Table 3: Results by instance size in the Paquay setting (30 instances per size): total volume of the ULDs (m^3) and number of ULDs ($|C|$) of the RGS and of BSG (mean of five runs).

	Volume of the ULDs (m^3)				$\lvert C\rvert$		
$\lvert I\rvert$	RGS	$t = 0.1\,\mathrm{s}$	$t = 1\,\mathrm{s}$	vs RGS	RGS	$t = 0.1\,\mathrm{s}$	$t = 1\,\mathrm{s}$
10	502.4	450.2	442.3	−12.0%	30	43	42
20	880.7	851.7	820.6	−6.8%	35	38.2	41
30	1128.5	1121.6	1034.4	−8.3%	47	55.8	51
40	1485.5	1520.3	1431.4	−3.6%	60	72	68.6
50	1735.4	1753.2	1674.3	−3.5%	66	75.6	73
60	2073.7	2024.7	1916.1	−7.6%	80	81.2	72
70	2278.5	2210.1	2188.7	−3.9%	82	94	88
80	2552.1	2503.4	2357.8	−7.6%	93	95.4	88.4
90	2805.4	2686.7	2623.7	−6.5%	102	111.2	104
100	3205.2	2984.3	2827.7	−11.8%	113	111.2	104.6

5.3 Comparison with a lower bound

A lower bound on the total volume of the ULDs is the smallest total volume of a set of ULDs whose volume and weight capacity suffice for the boxes and in which every box fits in some ULD, computed with the CP-SAT solver of OR-Tools. It assumes that the ULDs can be completely filled, which is far from possible with these boxes: in the Paquay setting the bound is $10\,141\,\mathrm{m}^3$, against $17\,317\,\mathrm{m}^3$ for our solutions with $t = 1\,\mathrm{s}$, whose ULDs are about 60% full. A simple bound on the number of ULDs (the total volume of the boxes divided by the volume of the largest ULD) is reached by our solutions in 90 of the 300 instances with $t = 1\,\mathrm{s}$, against 80 for the RGS, although the number of ULDs is only a secondary measure here.

5.4 Running time

With $t = 0.1\,\mathrm{s}$ our algorithm takes 0.56–0.89 s per instance on average (at most 7 s), and with $t = 1\,\mathrm{s}$ 3.5–6.1 s (at most 52 s). The RGS takes 0.19–0.42 s per instance on an Apple M2 Pro, about 0.5–1.1 s on our processor once its single-thread speed, about 2.5 times ours according to PassMark (PassMark Software), is taken into account. Table 5 and Figure 3 show how the volume changes with the time limit. At the time of the RGS our algorithm already uses about 2–5% less volume, and the difference grows with the time limit, up to 7–9% with $t = 2\,\mathrm{s}$; with a very short time limit ($t = 0.02\,\mathrm{s}$) it is slightly worse than the RGS. Giving the improvement phases the same time limit as the construction, instead of $0.1\,t$, gives better solutions for the same t but at a much higher running time: up to about 10 s per instance, the default is better at the same running time (Figure 3, Paquay setting).

Table 4: Results by instance size in the adapted settings (30 instances per size; see Table 3).

		Volume of the ULDs (m^3)				$\lvert C\rvert$		
	$\lvert I\rvert$	RGS	$t = 0.1$ s	$t = 1$ s	vs RGS	RGS	$t = 0.1$ s	$t = 1$ s
Adapted	10	497.3	433.0	433.0	−12.9%	30	43	43
	20	786.6	821.6	784.9	−0.2%	33	38	35
	30	1034.3	946.5	862.6	−16.6%	45	45.2	39
	40	1341.3	1324.4	1214.6	−9.4%	57	65.8	64
	50	1557.0	1555.1	1507.2	−3.2%	60	70.8	63
	60	1868.1	1813.1	1788.9	−4.2%	72	76.8	73
	70	2107.7	2002.9	1870.9	−11.2%	78	86	86.8
	80	2350.8	2244.0	2142.6	−8.9%	86	92.2	91
	90	2533.6	2392.8	2311.0	−8.8%	92	103.8	90
	100	2766.1	2669.2	2545.8	−8.0%	100	104.2	104.2
One	10	497.3	446.8	444.5	−10.6%	30	42	43
	20	786.6	804.8	768.7	−2.3%	33	38	34
	30	1034.3	950.7	866.8	−16.2%	45	44.2	38
	40	1341.3	1248.7	1196.5	−10.8%	57	64	63
	50	1538.2	1518.1	1414.5	−8.0%	62	73.6	65
	60	1825.7	1791.2	1758.3	−3.7%	77	81	81
	70	1996.6	1944.6	1891.8	−5.2%	85	93.8	88
	80	2192.9	2200.4	2126.0	−3.1%	97	99	98
	90	2358.6	2344.0	2258.2	−4.3%	109	109	105
	100	2594.2	2523.6	2481.6	−4.3%	134	132	130

Table 5: Effect of the time limit t (one run, five runs for $t = 0.1$ and 1 s): volume of the ULDs relative to the RGS, mean and maximum time per instance (s), and, with one ULD of each type, unloaded volume (m^3; RGS: 25.9).

	Paquay			Adapted			One			
t (s)	Vol.	Time	Max.	Vol.	Time	Max.	Vol.	Unl.	Time	Max.
0.02	+2.4%	0.24	2	+1.2%	0.29	1	+1.0%	40.9	0.34	3
0.05	−1.3%	0.35	2	−1.6%	0.46	2	−1.4%	18.3	0.55	4
0.1	−2.9%	0.56	4	−3.8%	0.77	4	−2.4%	20.0	0.89	7
0.2	−5.9%	0.94	5	−5.9%	1.32	5	−3.5%	10.7	1.50	13
0.5	−6.1%	1.92	14	−6.7%	2.77	12	−4.4%	15.5	3.26	31
1	−7.1%	3.51	16	−8.2%	5.15	20	−5.9%	17.8	6.13	52
2	−8.1%	6.64	32	−8.7%	9.62	39	−6.9%	19.6	11.52	93
RGS[a]		0.48			1.06			25.9	1.04	

[a] Mean time on the Apple M2 Pro scaled to our processor (factor 2.53).

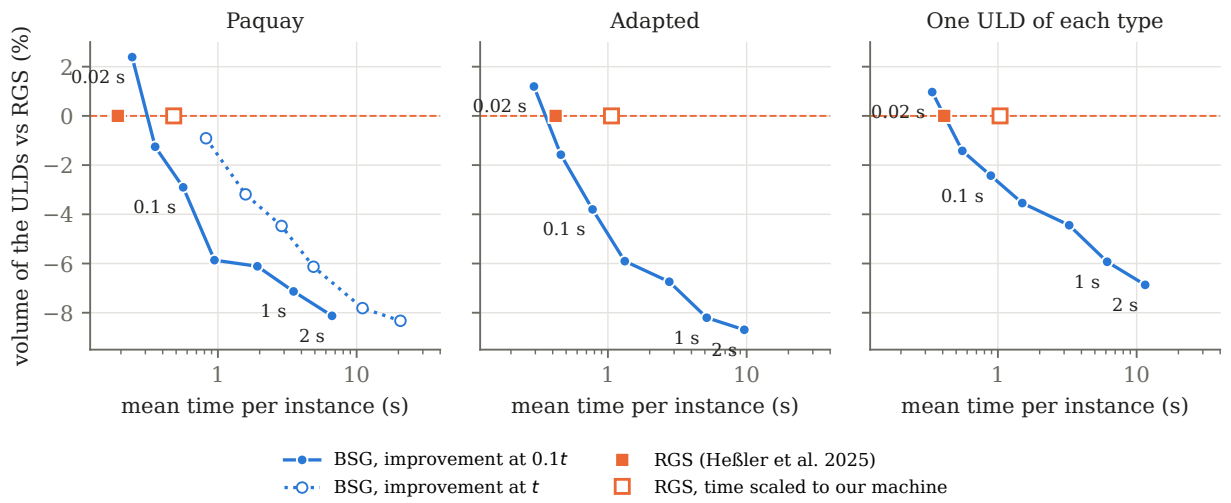


Figure 3: Volume of the ULDs relative to the RGS of Heßler et al. (2025) as a function of the mean time per instance (the labels are time limits t; mean of five runs for 0.1 s and 1 s, one run otherwise). The RGS is shown at its time on the Apple M2 Pro (filled square) and scaled to our processor by the single-thread speed ratio of 2.53 (empty square). In the Paquay setting, the dotted line gives the improvement phases the same time limit t as the construction.

5.5 Contribution of the components

Table 6 reports the results with $t = 0.1$ s when one component of the algorithm is removed (one run; the complete algorithm is the mean of five runs, whose volume varies by about 0.2%).

The reduction phase is the component that makes the difference with the RGS: without it, or without Insert, the volume is about the same as that of the RGS in the Paquay and adapted settings, and loading the leftover boxes into two new ULDs instead of one saves a further 0.2–1.3 points. The CoG penalty λ and the translation of the load keep the CoG in range: without either of them, 117–460 ULDs violate the constraint. Without the penalty the volume decreases in the Paquay setting, where the CoG limit of 5% is very strict, and increases in the other two; without the translation it increases in the three settings, since the penalty then rejects loadings that a translation would have balanced. Building the load with blocks of several boxes instead of single boxes reduces the volume by 1.4 points in the adapted setting and by less elsewhere, halves the CoG violations in the Paquay setting and reduces the unloaded volume with one ULD of each type: these boxes are highly heterogeneous, and few identical boxes can be grouped. The substructure saves 0.8–1.6 points in the adapted settings, and the restarts halve the unloaded volume with one ULD of each type (20.0 against 39.6 m^3). The choice of the ULD type with θ changes the volume by less than half a point in this run (0.8 points in the Paquay setting and with one ULD of each type in a run with $t = 0.05$ s): with an unlimited number of ULDs, a poor choice during the construction is mostly corrected by the reduction phase.

Other variants did not improve the algorithm at the same running time: penalizing the CoG excess in the evaluation of the blocks (its effect on the volume changed sign between runs), anchoring the spaces at the nearest corner of the ULD (less volume in the Paquay setting, but with 50% more time, at which a larger t is better), reloading each ULD from scratch or with priority in Insert, eliminating ULDs without replacing them, and choosing the type of each ULD by completing the solution with a fast construction (a rollout).

Table 6: Ablation ($t = 0.1\,\mathrm{s}$): volume of the ULDs relative to the RGS, ULDs with the CoG out of range in the length or width direction and, with one ULD of each type, unloaded volume (m^3). In bold, the measures that get clearly worse than with the complete algorithm. Basic BSG: rule of Heßler et al. (2025), spaces anchored at the nearest corner of the ULD, no substructure, no restarts and no reduction phase.

	Paquay		Adapted		One		
Variant	Vol.	CoG	Vol.	CoG	Vol.	Unl.	CoG
Complete algorithm	−2.9%	7.8	−3.8%	3	−2.4%	20.0	17
No CoG penalty ($\lambda = 0$)	−6.0%	**460**	−3.0%	**281**	−1.0%	13.5	**326**
No translation of the load	**−1.4%**	**183**	−2.4%	**117**	−1.9%	**36.7**	**216**
Single-box blocks ($N_{\max} = 1$)	−2.7%	**16**	**−2.4%**	4	−2.3%	**24.0**	21
No substructure	–	–	**−2.2%**	7	**−1.6%**	17.5	23
No restarts	–	–	–	–	−2.4%	**39.6**	15
Rule of Heßler et al. (2025) (no θ)	−2.5%	7	−3.6%	3	−2.7%	20.3	18
No Insert	**+0.2%**	10	**+0.2%**	4	**−1.2%**	**37.1**	21
Reduction with one new ULD	**−1.6%**	8	**−2.5%**	2	−2.2%	20.0	16
No reduction phase	**+0.8%**	10	**+0.6%**	2	**−1.0%**	20.0	20
Basic BSG	**+13.9%**	**90**	**+18.4%**	**61**	**+10.1%**	**43.0**	**72**

5.6 Volume of the ULDs versus weight distribution

Table 7 shows the effect of the penalty λ of the CoG excess. With $\lambda = 0$ the CoG is ignored and 279–462 ULDs violate the constraint; any $\lambda \geq 0.5$ reduces the violations to at most 33 ULDs, and $\lambda \geq 2$ to at most 14, with a volume that changes by at most 1.6 points. The constraint can therefore be enforced almost completely at a small cost in volume, and the tuned value $\lambda = 1.4$ lies in this flat region.

Table 7: Effect of the CoG penalty λ ($t = 0.1\,\mathrm{s}$, one run; $\lambda = 1.4$ is the complete algorithm, mean of five runs): volume of the ULDs relative to the RGS and ULDs with the CoG out of range in the length or width direction.

	Paquay		Adapted		One	
λ	Vol.	CoG	Vol.	CoG	Vol.	CoG
0	−6.0%	462	−3.0%	279	−1.0%	326
0.5	−3.3%	20	−4.4%	8	−2.4%	33
1	−2.2%	14	−4.0%	5	−2.7%	22
1.4	−2.9%	7.8	−3.8%	3	−2.4%	17
2	−3.5%	7	−3.5%	1	−2.3%	14
5	−3.7%	6	−3.7%	0	−2.6%	11
30	−3.8%	6	−4.0%	0	−2.6%	2

6 Conclusions

We have adapted the block-based beam search BSG, originally designed for the single container loading problem, to the palletization of air cargo into ULDs with non-cuboid shapes, a floor edge and a substructure, weight capacities, fragile boxes, support constraints and CoG constraints. The ULD constraints are handled in the generation and placement of the blocks, and the CoG constraint

in the value of the solutions and with a final translation of the load, which is kept grouped by anchoring the free spaces at one corner of the ULD. The ULDs are loaded sequentially, and the total volume of the ULDs is then reduced with a single move that removes a ULD, inserts its boxes into the others and loads the rest into at most two smaller ULDs.

On the 300 instances of Paquay et al. (2018a), and with the same parameters in all the settings, our algorithm uses 2.9–7.1% less ULD volume than the RGS of Heßler et al. (2025) in the original setting, 3.8–8.2% less in the adapted one and 2.4–5.9% less with one ULD of each type, the smaller savings at a comparable running time; it also leaves less volume unloaded, and almost none of its ULDs violates the CoG constraint. The experiments also show that the choice of the objective matters: minimizing the number of ULDs leads to fewer but larger ULDs, and the volume is mainly reduced by moves that look at the whole solution; preferring smaller ULD types during the construction helps, but much less.

These results show that the ideas that made block-based tree search successful in container loading, namely building the load with blocks of boxes, evaluating them with the VCS function and guiding a beam search with greedy completions, carry over to highly heterogeneous air cargo, provided that the blocks and their placement respect the support and fragility constraints and that the weight distribution is taken into account during the search.

Several directions remain open. A stronger lower bound would tell how far the remaining solutions are from the optimum. The sequential framework could be replaced by a search that decides the assignment of the boxes to the ULDs jointly, which matters most when the number of ULDs is limited. Other practical constraints, such as load bearing capacities, the grouping of the boxes by destination or the balance of the whole aircraft, could also be integrated into the evaluation of the blocks and of the solutions.

References

Ramón Alvarez-Valdes, Francisco Parreño, and José Manuel Tamarit. A GRASP/Path Relinking algorithm for two- and three-dimensional multiple bin-size bin packing problems. *Computers & Operations Research*, 40(12):3081–3090, 2013.

Ignacio Araya and María-Cristina Riff. A beam search approach to the container loading problem. *Computers & Operations Research*, 43:100–107, 2014.

Ignacio Araya, Keytel Guerrero, and Eduardo Nuñez. VCS: A new heuristic function for selecting boxes in the single container loading problem. *Computers & Operations Research*, 82:27–35, 2017.

Ignacio Araya, Mauricio Moyano, and Cristobal Sanchez. A beam search algorithm for the biobjective container loading problem. *European Journal of Operational Research*, 286(2):417–431, 2020.

Mauro Maria Baldi, Guido Perboli, and Roberto Tadei. The three-dimensional knapsack problem with balancing constraints. *Applied Mathematics and Computation*, 218(19):9802–9818, 2012.

Eberhard E. Bischoff and M. S. W. Ratcliff. Issues in the development of approaches to container loading. *Omega*, 23(4):377–390, 1995.

Andreas Bortfeldt and Gerhard Wäscher. Constraints in container loading – a state-of-the-art review. *European Journal of Operational Research*, 229(1):1–20, 2013.

Felix Brandt and Stefan Nickel. The air cargo load planning problem – a consolidated problem definition and literature review on related problems. *European Journal of Operational Research*, 275(2):399–410, 2019.

Felix T. S. Chan, Rajat Bhagwat, Nishant Kumar, Manoj K. Tiwari, and P. Lam. Development of a decision support system for air-cargo pallets loading problem: A case study. *Expert Systems with Applications*, 31(3):472–485, 2006.

Teodor Gabriel Crainic, Guido Perboli, and Roberto Tadei. Extreme point-based heuristics for three-dimensional bin packing. *INFORMS Journal on Computing*, 20(3):368–384, 2008.

A. Paul Davies and Eberhard E. Bischoff. Weight distribution considerations in container loading. *European Journal of Operational Research*, 114(3):509–527, 1999.

Tobias Fanslau and Andreas Bortfeldt. A tree search algorithm for solving the container loading problem. *INFORMS Journal on Computing*, 22(2):222–235, 2010.

Oluf Faroe, David Pisinger, and Martin Zachariasen. Guided local search for the three-dimensional bin-packing problem. *INFORMS Journal on Computing*, 15(3):267–283, 2003.

Mikele Gajda, Alessio Trivella, Renata Mansini, and David Pisinger. An optimization approach for a complex real-life container loading problem. *Omega*, 107:102559, 2022.

Lucas Guesser, Thiago Alves de Queiroz, and Flávio Keidi Miyazawa. A tree search-based heuristic for the three-dimensional single container loading problem. *European Journal of Operational Research*, 333:337–348, 2026.

Katrin Heßler, Timo Hintsch, and Lukas Wienkamp. A fast optimization approach for a complex real-life 3D Multiple Bin Size Bin Packing Problem. *European Journal of Operational Research*, 2025. arXiv:2410.01445.

Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

Virginie Lurkin and Michael Schyns. The airline container loading problem with pickup and delivery. *European Journal of Operational Research*, 244(3):955–965, 2015.

Silvano Martello, David Pisinger, and Daniele Vigo. The three-dimensional bin packing problem. *Operations Research*, 48(2):256–267, 2000.

A. C. P. Mesquita and C. A. A. Sanches. Air cargo load and route planning in pickup and delivery operations. arXiv:2606.26404, 2026.

Frank G. Ortmann, Nthabiseng Ntene, and Jan H. van Vuuren. New and improved level heuristics for the rectangular strip packing and variable-sized bin packing problems. *European Journal of Operational Research*, 203(2):306–315, 2010.

Céline Paquay, Michael Schyns, and Sabine Limbourg. A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application. *International Transactions in Operational Research*, 23(1-2):187–213, 2016.

Céline Paquay, Sabine Limbourg, and Michael Schyns. A tailored two-phase constructive heuristic for the three-dimensional Multiple Bin Size Bin Packing Problem with transportation constraints. *European Journal of Operational Research*, 267(1):52–64, 2018a.

Céline Paquay, Sabine Limbourg, Michael Schyns, and José Fernando Oliveira. MIP-based constructive heuristics for the three-dimensional Bin Packing Problem with transportation constraints. *International Journal of Production Research*, 56(4):1581–1592, 2018b.

Francisco Parreño, Ramón Alvarez-Valdes, José Manuel Tamarit, and José Fernando Oliveira. A maximal-space algorithm for the container loading problem. *INFORMS Journal on Computing*, 20(3):412–422, 2008.

Francisco Parreño, Ramón Alvarez-Valdes, José Fernando Oliveira, and José Manuel Tamarit. A hybrid GRASP/VND algorithm for two- and three-dimensional bin packing. *Annals of Operations Research*, 179(1):203–220, 2010.

PassMark Software. CPU Benchmarks: single thread performance. https://www.cpubenchmark.net. Accessed October 2026.

Gerhard Schrimpf, Johannes Schneider, Hermann Stamm-Wilbrandt, and Gunter Dueck. Record breaking optimization results using the ruin and recreate principle. *Journal of Computational Physics*, 159(2):139–171, 2000.

Alessio Trivella and David Pisinger. The load-balanced multi-dimensional bin-packing problem. *Computers & Operations Research*, 74:152–164, 2016.

Gerhard Wäscher, Heike Haußner, and Holger Schumann. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 183(3):1109–1130, 2007.

Xiaozhou Zhao, Julia A. Bennell, Tolga Bektaş, and Kath Dowsland. A comparative review of 3D container loading algorithms. *International Transactions in Operational Research*, 23(1-2): 287–320, 2016.

Wenbin Zhu and Andrew Lim. A new iterative-doubling Greedy–Lookahead algorithm for the single container loading problem. *European Journal of Operational Research*, 222(3):408–417, 2012.